1 **Design & Implementation of Human-Computer Interfaces**
NPTEL-MOOCS

Dr Samit Bhattacharya
Computer Science and Engineering
IIT Guwahati

2 **L18 – Converting Design to System: System Design Basics**

Dr Samit Bhattacharya
Computer Science and Engineering
IIT Guwahati

3

4 **Recap**

- Earlier we learned about interface design
  - Issues and guidelines
- Today we shall start discussion on how to convert the interface design to system design

5 **System Design**

6 **Note**

- The design-prototype-evaluate cycle primarily caters to interface design
- Once interface design stabilized and no further iteration required, we focus on system design

7 **Design Concerns**

- Two issues
  - Where to start?
  - How to represent (design language)?

8 **Main Idea**

- To make the code "manageable"

9 **Where to Start?**

- SRS – Software Specification Language
- Two phases
  - Preliminary (high-level) design
  - Detailed design (also called module-specification document)

10 **High-Level Design**

- Identification of modules
- Control relationships between modules
- Definition of interfaces between modules

11 **How to Represent?**
- Many "languages"

12

13

14

15 **Detailed Design**
- Identification of data structures and algorithms for different modules

16 **Characteristics of Good Design**
- Coverage – should implement ALL functionalities of SRS
- Correctness - should CORRECTLY implement all functionalities of SRS
- Understandability - easily understandable (by other team members)

17 **Characteristics of Good Design**
- Efficiency - should be efficient (in terms of resources required to implement)
- Maintainability – should be amenable to change

18 **Cohesion & Coupling**

19 **"Modular" Design**
- Good software design requires
  - Clean decomposition of the problem into modules
  - Neat arrangement of modules in a hierarchy
- Modularization depends on *cohesion & coupling*

20 **Cohesion (of a Module)**
- Logical – if all functions perform similar operations (e.g., error handling)
- Temporal – if all functions should be performed in the same time span (e.g., initialization module)
- Procedural/functional – if all functions are part of the same procedure (algorithm) (e.g., decoding algorithm)

21 **Cohesion (of a Module)**
- Communication – if all functions refer to or update same data structure (e.g., a set of functions operating on a linked list)

- Sequential – output from one element is input to the next element of the module (e.g., the sequence of functions get-input, validate-input, sort-input)

22 **Coupling (between Modules)**

- Data – if two modules communicate through a data item (e.g., passing an integer between two modules)
- Control – if data from one module is used to control the flow of instructions in the other module (e.g., flag setting)
- Content – if two modules share code (e.g., branch from one module to another)

23 **Cohesion and Coupling**

- High cohesion & low coupling ☐ functionally independent modules

24 **Basic Code Design Approaches**

- TWO approaches

25 **Basic Design Approaches**

- Function oriented – basic abstractions are functions
  - Use DFD to represent design

26 **Basic Design Approaches**

- Object oriented – basic abstractions are objects (instantiation of class)
  - Use UML to represent design

27 **Note**

- We shall learn about both in subsequent lectures – with case studies